

Context Minimization as a Lens for Software Design

Leric Zhang

2026-04-24

Software design principles are often taught as separate rules, making it difficult to explain why they help, when they conflict, and when following them becomes over-engineering. This position paper proposes the **Context Minimization Principle** (CMP): a design is better, all else equal, when it reduces the context a human developer or AI coding agent must load to answer a concrete engineering query reliably, without obscuring essential domain complexity. Although engineering queries are diverse, many maintenance tasks cluster around two recurring forms: **unit-or-path understanding**, which asks what a code unit or execution path does, and **purpose change**, which asks where a shifted requirement must be edited. CMP maps these query clusters to two dominant modes of context expansion: **depth**, through dependency traversal, and **breadth**, through dispersed location search. Their trade-off yields an **Over-engineering Criterion**: a design move becomes suspect when the depth it introduces is not justified by the breadth it eliminates for the realistic engineering queries a system must support. Through this lens, familiar design principles and programming language abstractions become context-control mechanisms rather than isolated prescriptions.

1 Introduction

Software engineering has accumulated a rich vocabulary of design advice: information hiding, modularity, coupling and cohesion, DRY, SOLID, domain-driven design, clean architecture, test-driven development, and many others. These principles are useful, but they are usually taught as separate rules. When they agree, this is not a problem. When they conflict, practitioners often fall back on experience, local taste, or slogans.

The conflicts are familiar. DRY discourages duplication, but the wrong abstraction can make simple behavior harder to understand. Clean architecture protects domain logic from infrastructure, but excessive layering can force a reader through multiple files before any

behavior is visible. Dependency inversion can stabilize a boundary, but a forest of interfaces can turn every change into dependency-graph archaeology. These tensions suggest that software design needs a common currency for comparing design moves. Code size, dependency count, and abstraction level are useful proxies, but they do not directly capture the cost developers most directly experience: the context required to answer an engineering question correctly.

This paper proposes the **Context Minimization Principle** (CMP) as that currency:

For a given engineering query, a design is better, all else equal, when it reduces the amount of relevant context that a human developer or AI coding agent must load to answer the query reliably, without obscuring essential domain complexity.

CMP is not a call for less information per se. Software carries essential complexity that cannot be removed (Brooks 1987), and hiding essential information behind unreliable boundaries merely relocates confusion. CMP targets *avoidable* context: information a reader must load only because design decisions are scattered, leaky, duplicated, implicit, or poorly bounded.

The key move is to treat context as **query-relative** without making design judgment arbitrary. A code unit is not intrinsically high-context or low-context. It has a context footprint for a particular question: adding a field, verifying an invariant, tracing a production value, reviewing a refactor, or extending a domain rule.

Although engineering queries are diverse, many everyday maintenance tasks can be usefully approximated by two recurring clusters: *unit-or-path understanding* - “what does this code, or this execution path, actually do?” - and *purpose change* - “the requirement has shifted; where must I edit?” These clusters principally call upon two modes of context expansion: **depth**, which expands along dependency chains, and **breadth**, which expands across dispersed locations.

The mapping is directional rather than exclusive: most queries exercise both modes in some mix. The analytical unit of CMP is therefore the *(depth, breadth) profile* a design imposes on a query and, over a realistic query distribution, on the workload as a whole. This gives CMP operational bite. It does not retreat to “it depends”; it asks what *(depth, breadth) profile* a design move produces for the queries the system actually faces.

This paper is a position paper: it proposes the theoretical structure that any concrete context metric must satisfy, rather than a specific metric or tool. The Research Agenda section discusses how depth and breadth might be operationalized in practice and what would count as empirical validation.

The paper makes three contributions:

1. It defines CMP as a query-relative framework for reasoning about avoidable context in software design.

2. It identifies two recurring modes of context expansion - **depth** (dependency traversal) and **breadth** (location search) - and relates them to two common maintenance query clusters: *unit-or-path understanding* and *purpose change*.
3. It formulates the **Over-engineering Criterion** from the depth-breadth trade-off: a design move is justified only when the context it removes is worth the context it adds for the realistic engineering queries the system must support.

The rest of the paper develops these ideas and then uses them to reinterpret selected design principles, programming language abstractions, and AI-assisted software engineering as instances of context control.

2 The Context Minimization Principle

2.1 Context as a Query-Relevant Working Set

In this paper, an **engineering query** is a concrete question posed against a codebase with a definable correct answer or success criterion. Examples include:

- *Unit-or-path understanding*: What does this code path actually do?
- *Purpose change*: What must change to add this field end-to-end?
- *Verification*: Will changing this dependency violate an invariant?
- *Debugging*: Where does this unexpected production value come from?

Drawing an intuition from information theory - where information is defined in relation to uncertainty reduction (Shannon 1948) - an engineering query represents a specific state of uncertainty. **Context**, then, is the sufficient working set of signals a reader must process to reach a reliable answer. It represents the effective scope of code, configuration, tests, documentation, types, runtime conventions, and domain assumptions that must be loaded for the query at hand.

Crucially, context can be **explicit** or **implicit**. Explicit context is mechanically recoverable: a type signature, a well-named interface, or an inline comment. Implicit context consists of unstated assumptions, historical conventions, and “tribal knowledge” missing from the source text. For example, consider a module that requires `init()` to be called before `process()`. If this ordering is merely a convention, the temporal coupling is implicit context - a reader must trace the entire call stack to verify the rule is respected. If the design instead mandates that `init()` returns a `ReadyState` token which `process()` explicitly requires as an argument, the implicit historical knowledge is transformed into localized, compiler-verified context. A primary function of good software design is to convert unbounded implicit context into localized explicit context.

Since context is the working set needed to answer a particular engineering query, it is **query-relative**, not an intrinsic property of the code itself. Consider a function that calculates a

discount for an order. For the query “*does this function apply the threshold correctly?*”, the required context may be limited to the function body and its tests. For the query “*why does this call produce this discount?*”, the reader may need to follow the execution path that selects and applies the rule. For the query “*what must change to replace the discount policy?*”, the reader may instead need to find every representation of that policy across the system. The code under inspection may be the same, but the relevant context changes with the question being asked.

This relativity does not make CMP vacuous. In practice, maintenance work is dominated by recurring query families. The first is **unit-or-path understanding**: *what does this code, or this execution path, actually do?* The second is **purpose change**: *the requirement has shifted; where must I edit?* These two query clusters motivate the two principal modes of context expansion used throughout this paper: **depth** and **breadth**.

2.2 Depth and Breadth Profiles

Depth refers to context expansion through dependency traversal. It is most visible in unit-or-path understanding queries. A developer begins with a code unit, but understanding it requires following calls, inheritance chains, configuration bindings, framework callbacks, global state, generated code, or runtime conventions. The deeper the traversal, the more context must be loaded before local reasoning becomes reliable.

For example, imagine trying to understand what discount a call to `calculateDiscount(order, customer)` will actually apply. In a deeply abstracted pricing system, answering this query might require following a `PricingService` into a `DiscountPolicy` interface, then into a strategy factory, promotion configuration provider, feature-flag check, and concrete rule implementation before the behavior becomes visible. The query begins at one code path, but reliable understanding requires dependency traversal.

Depth is introduced whenever a design places behavior behind another boundary: a function call, module interface, class hierarchy, framework hook, configuration layer, dependency-injected service, or language-level abstraction. This is not inherently bad. Abstraction almost always adds some dependency traversal, but it can still reduce total context when the boundary gives the reader a reliable stopping point. A well-named function, a precise type signature, a stable module interface, or a tested policy object may add one step of depth while preventing much larger traversal into scattered implementation details.

Depth becomes costly when the added traversal does not stop soon enough. This happens when boundaries are vague, contracts are incomplete, runtime binding is hard to inspect, configuration changes behavior implicitly, inheritance or callback chains obscure control flow, or callers must repeatedly inspect implementation details to use the abstraction safely. The design goal for depth is therefore not to avoid abstraction, but to make added depth cheap: a boundary is reliable when the client can use it without repeatedly traversing into its internals.

Type signatures, tests, documentation, invariants, module visibility, and stable interfaces can all help create such boundaries.

Breadth refers to context expansion through location search and knowledge dispersion. It is most visible in purpose-change queries. Instead of following a deep chain, the developer must search across many locations because one piece of knowledge is duplicated, fragmented, or inconsistently represented.

For example, consider a requirement to change a discount rule from a flat percentage to a tiered policy based on order value and customer segment. If the rule is encoded separately in the checkout UI, backend order service, billing job, and reporting pipeline, the developer must find and update each representation. The knowledge of how discounts are calculated is dispersed, forcing a broad and fragile context search.

Common sources of breadth include duplicated business rules, copy-pasted validation logic, inconsistent naming, distributed configuration, schema drift, parallel implementations, and similar domain concepts represented differently in different layers. Breadth is especially costly during modification: changing one rule requires discovering all places where the rule is encoded.

The design goal for breadth is to keep each piece of knowledge localized enough that change does not require broad search and synchronization. This is the context-based rationale behind DRY, single source of truth, schema centralization, shared domain language, and bounded contexts. The point is not to eliminate every repeated token. The point is to avoid dispersing the same decision across places that must evolve together.

Depth-breadth profiles. The mapping from query cluster to context mode is directional rather than exclusive. Understanding a behavior may require some breadth if the behavior is implemented in several places. Changing a business rule may require depth if the authoritative rule is hidden behind indirection. The analytical unit of CMP is therefore not a scalar context score, but a **depth-breadth profile**: the combination of dependency traversal and location search a design imposes on a query and, over a realistic query distribution, on the workload as a whole.

2.3 The Principle

The **Context Minimization Principle** can be stated more formally:

A design is better, all else equal, when it reduces the avoidable context required to answer the engineering queries the system must realistically support, while faithfully representing the domain complexity those queries depend on.

Because context is query-relative, CMP evaluates designs against the engineering queries a system realistically supports, not against simplicity in the abstract. This also means that CMP is not equivalent to code minimization: shorter code or fewer abstractions can increase context

when they replace explicit structure with implicit convention, scattered decisions, or unreliable boundaries.

In practice, CMP is often achieved not by eliminating context outright, but by restructuring it: reorganizing information so that relevant queries require less dispersed search, while using reliable boundaries to keep any added traversal manageable.

2.4 The Over-engineering Criterion

Depth and breadth capture different failure modes: a design can be locally easy to understand while still expensive to change, or centralized enough to avoid search while still difficult to follow. For the discount-policy workload, one design may impose low depth but high breadth. Each individual implementation may be locally straightforward, but the discount rule might be independently encoded in the checkout UI, backend order service, billing job, and reporting pipeline. A purpose-change query such as changing the rule safely now requires broad, exhaustive search and synchronization. We will call such a design **knowledge-scattered**.

A design can also impose high depth and low breadth. The same discount rule might be defined in exactly one place, perfectly honoring the DRY principle. However, reaching that place might require traversing a generic `RuleEngine` interface, unpacking a dependency-injected `DiscountStrategyFactory`, deciphering a chain of abstract decorators, and finally locating the concrete implementation registered only at runtime. The knowledge is not duplicated, yet the path to it is labyrinthine. We will call such a design **deeply abstracted**.

Between these extremes, an intermediate design might define a `DiscountPolicy` or `PricingPolicy` with a clear contract, such as `calculateDiscount(order, customer)`. This adds some depth: callers must understand the policy boundary. But it can remove much more breadth by giving the discount decision an authoritative representation. This is the useful case of abstraction under CMP.

These cases lead to the operational judgment rule of CMP:

Over-engineering Criterion. A design decision is over-engineered when the effective depth it adds is not justified by the breadth it eliminates for the realistic engineering queries the system must support.

The criterion treats abstraction, layering, configuration, and indirection as context trade-offs rather than inherently good or bad moves. Many structural design moves reduce breadth by placing related information behind a boundary, accepting some additional depth in return. They improve the design when the boundary is reliable enough that the added traversal remains cheap; they become over-engineering when the added depth dominates the breadth actually removed.

Several familiar cases follow directly:

- **Premature abstraction** pays depth cost before there is enough breadth pressure to justify it.
- **Rule of Three.** When duplication has occurred only once or twice, breadth cost is low, and the depth cost of a shared abstraction often dominates. This gives a context-cost reading of Sandi Metz’s observation that “duplication is far cheaper than the wrong abstraction” (Metz 2016).
- **Over-layered architecture** forces queries to traverse layers whose breadth benefits do not, in practice, materialize.
- **Over-parameterization** adds depth - every reader must reason across configuration branches - while producing breadth benefits only if multiple instantiations genuinely exist.
- **Duplication of simple, stable logic** can be justified when the breadth cost is lower than the depth cost of a shared abstraction.

Because this trade-off is the primary driver of design friction, any attempt to measure codebase quality - whether through static analysis or AI agent metrics - must account for both depth and breadth. Optimizing only one will systematically mislead design decisions.

3 Applications: Design Principles as Context Controls

The purpose of CMP is not to replace existing design principles with new slogans. It is to explain why familiar principles help, why they conflict, and where they stop helping. The examples below are therefore selective rather than encyclopedic. Each illustrates a different context-control mechanism: making depth cheap, reducing breadth, trading breadth for depth, or turning implicit assumptions into explicit checks.

3.1 Information Hiding: Making Depth Cheap

Parnas argued that modules should be decomposed around design decisions likely to change, hiding those volatile decisions behind stable interfaces (Parnas 1972). Ousterhout later described a similar ideal as “deep modules”: small interfaces that conceal substantial implementation complexity (Ousterhout 2018). In CMP terms, information hiding makes **depth** cheaper. It does not eliminate the existence of a dependency; it creates a boundary at which dependency traversal can stop. A good boundary lets a caller answer a unit-or-path understanding query without traversing into representation details.

This also clarifies why superficial encapsulation is not enough. Making fields **private** while exposing setters does not reduce context if callers still need to understand the object’s internal state machine to use those setters safely. The boundary only works when it changes the query’s depth profile: the caller can rely on an explicit contract instead of loading implementation history.

3.2 DRY and Knowledge Locality: Cutting Breadth

DRY is best read as a breadth-control principle. When the same business decision is encoded in multiple locations, a purpose-change query becomes a search problem: every representation of the decision must be found, understood, and updated consistently. Cohesion, single source of truth, schema centralization, and DDD’s bounded contexts and ubiquitous language all serve a similar role: they reduce the number of places a reader must search to understand or change a domain concept (Evans 2003). In depth-breadth terms, DRY typically reduces breadth by introducing depth through a shared abstraction. It helps only when that abstraction becomes a reliable boundary; otherwise it merely replaces location search with dependency traversal.

The context lens also explains why DRY is frequently misapplied. Two similar-looking loops in unrelated domains may be structural duplication but not knowledge duplication. Extracting them into a shared abstraction removes little breadth while adding depth for every caller. This is the context-cost version of the warning that “duplication is far cheaper than the wrong abstraction” (Metz 2016). DRY helps when it localizes a decision that must evolve together; it hurts when it invents a shared concept that the domain does not actually share.

3.3 Architecture: System-Level Context Boundaries

At the architectural level, context control is less about a single abstraction and more about the placement of boundaries across the system. Clean and hexagonal architecture separate domain logic from infrastructure by planning which context regions may depend on which others: domain policy, use cases, persistence, delivery mechanisms, and external services (Cockburn 2005; Martin 2017). These boundaries act as a map for where different kinds of decisions should live.

Such architectures still introduce depth: readers must understand ports, adapters, use cases, and dependency rules. Their value comes from making both traversal and search systematic rather than accidental. For example, when a discount rule changes, a good architecture lets the reader reason about where the relevant decision should live: in the domain policy or use-case layer, not in HTTP handlers, ORM mappings, payment-provider adapters, or scheduled jobs. The reader may still need to inspect the policy implementation, but the architecture turns an open-ended search problem into a directed lookup. In CMP terms, architecture controls context by making the expected location of decisions inferable from the system’s boundary plan.

The failure mode is not that architectural boundaries mechanically add depth, but that the boundary plan no longer predicts where the answers to important queries should live. A port with no behavioral contract forces readers back into adapter implementations; a layer around trivial CRUD behavior regularizes code without protecting any important query; a boundary drawn in the wrong place can scatter one domain decision across multiple architectural regions.

3.4 Tests: Making Context Explicit

Tests, especially in the TDD tradition, turn implicit operating assumptions into executable context (Beck 2002). A focused test does not only state what must remain true; it also shows what context must be supplied for the code to run: inputs, collaborators, configuration, persisted state, time, permissions, or domain preconditions. In order to test a function, the test must construct the external context that the function depends on. When that setup is small and explicit, future readers can understand the code's working conditions locally. When the setup requires hidden globals, broad fixtures, temporal ordering, or framework magic, the test exposes context that the design has failed to make explicit.

This is why tests can improve design rather than merely check it. By forcing required context to be constructed explicitly, tests pressure a design to move dependencies into parameters, constructors, ports, types, or well-named fixtures. A function that is hard to test is often hard to understand for the same reason: too much of its required context is implicit, ambient, or scattered outside the call boundary. In this sense, tests make boundaries more reliable in two ways: they specify what behavior a reader can depend on, and they reveal what context must be present for that behavior to hold.

Tests can also inflate context. Brittle tests that mock internal details encode the *how* rather than the *what*. Overgrown fixtures can make a small behavior depend on a large artificial world. When a harmless refactor breaks many tests, or when understanding a test requires reconstructing the implementation structure it mirrors, the tests have stopped acting as boundaries and have become another dispersed representation of implementation context. In CMP terms, good tests localize required behavioral context; bad tests duplicate internal context as breadth.

3.5 Other Heuristics and Patterns

The same lens can be extended to many other design vocabularies, but the mappings should be read as heuristic rather than exhaustive. CMP does not replace these principles or reduce them to a single theory. It provides a comparative lens for asking what context a design practice removes, what context it adds, how it mediates conflicts between competing principles, and which choice best fits the engineering queries a system actually faces.

Several examples are worth including only insofar as CMP gives them a non-trivial reading:

- **Coupling and cohesion.** CMP shifts the discussion from structural edge counts to query radius. A dependency is costly when answering a local question requires importing another component's conceptual model; cohesion is valuable when the facts needed for one engineering query tend to live within the same working set. In this reading, coupling and cohesion shape the topology of context expansion, not merely the number of module relationships.

- **Single responsibility principle.** SRP is hard to apply because *responsibility* is fuzzy - writers naturally conflate it with “one feature point.” Robert Martin’s “one reason to change” already shifts the test from writing to modification; CMP sharpens it further by exposing that modification cost has two axes, not one. A unit can fail SRP on the **depth** axis, when opening it forces the reader to hold unrelated working sets at once, or on the **breadth** axis, when a single purpose-change query has to touch many units because the responsibility was scattered. A well-bounded unit is right on both at once: inside speaks one coherent vocabulary, outside attracts one coherent class of purpose-change queries.
- **Dependency inversion and inversion of control.** DI and IoC are valuable because they convert ambient context into declared context. Instead of requiring a reader or test to discover hidden collaborators, configuration, or framework state, the component states what must be supplied for it to work. More importantly, the declaration is made from the consumer’s need: the component describes the capability it needs to perform its role, rather than choosing from whatever surface a provider happens to expose. In CMP terms, understanding this component becomes a local task: every external collaboration it relies on already appears at the boundary in the shape the component needs, so a reader can answer what the unit does by reading the unit itself, without leaving it to study how those collaborators are implemented.
- **Facade pattern.** A facade acts as a context firewall. It presents the caller with the smallest task-shaped surface that is meaningful for the caller’s query, while keeping raw APIs, incidental capabilities, and subsystem-specific domain knowledge behind the boundary. The point is not just simplification, but preventing the caller from having to learn a richer conceptual model than the task requires.
- **CQRS and read models.** CQRS is often motivated by persistence, scaling, and query-performance concerns, but its CMP reading is about separating two kinds of context. The command side preserves the context needed to validate invariants and state transitions; read models remove that mutation context from query code by projecting state into a purpose-built representation. The design helps when read queries and write correctness genuinely require different working sets.
- **Event-driven design.** Event-driven design does not eliminate context; it changes its shape. It replaces call-stack reasoning with causal-flow reasoning: instead of asking which callee runs next, the reader asks what fact was published, who observes it, in what order, and with what failure semantics. It reduces direct dependency depth only when event contracts, ownership, and traces make that causal context discoverable.

These examples share the same shape without requiring every practice to have a symmetrical failure mode. CMP asks whether a practice improves the depth-breadth profile of the queries the system actually faces, not whether the practice is categorically good or bad.

4 Programming Language Abstractions as Enforced Context Boundaries

The preceding sections introduced context control as a depth-breadth trade-off. One recurring mechanism behind that trade-off is the **boundary**: a point where context expansion can stop because the reader can rely on a contract rather than loading the implementation, history, or convention behind it. For CMP, the value of a boundary depends on its reliability. A weak boundary sends the reader back across it; a strong boundary lets the reader reason locally.

Among the boundary-making tools available to software designers, programming language mechanisms are distinctive because they can make some design constraints mechanically enforceable. Types, modules, visibility rules, ownership, and effect annotations can express boundary contracts directly in source code, where a compiler or runtime can check them. In CMP terms, they turn context that would otherwise be remembered, inferred, or manually verified into local contracts that other design abstractions can rely on.

4.1 Examples of Language-Supported Boundaries

The following table is illustrative rather than taxonomic. It shows recurring ways in which programming language mechanisms can strengthen design boundaries:

Language mechanism	Boundary strengthened	Context controlled	How it reduces context
Type signatures and stronger type systems	Value boundary	Value and shape context	Callers can rely on declared input and output constraints rather than inferring them from implementations or usage patterns.
Visibility and module systems	Reachability boundary	Naming, dependency, and representation context	Internals that should not matter to a query are made inaccessible or non-importable, limiting accidental dependence on hidden details.

Language mechanism	Boundary strengthened	Context controlled	How it reduces context
Interfaces, traits, and abstract types	Behavioral boundary	Substitutability and capability context	Clients can depend on a declared contract rather than concrete implementations, provided the contract is precise enough to be relied upon.
Explicit error handling and effect systems	Effect boundary	Failure and side-effect context	Possible failures or effects become part of the local contract instead of hidden control flow discovered by call-stack traversal.
Immutability and value semantics	Mutation boundary	State-history context	Readers need not ask who else may mutate the value after it is observed.
Ownership and borrowing	Aliasing boundary	Lifetime, aliasing, and resource context	Resource and mutation constraints are checked locally instead of reconstructed from non-local conventions.
Exhaustive pattern matching	Variant boundary	Case-completeness context	Readers and tools can know whether all variants of a domain state have been handled.

These examples are not mutually exclusive or exhaustive. The recurring pattern is that a feature helps CMP when it turns an implicit question into an explicit boundary. Most such mechanisms primarily reduce **depth** by letting traversal stop at a checked contract. Some also reduce **breadth** by preventing the same invariant from being manually re-expressed across many locations. The stronger and more precise the boundary, the less dependency traversal or location search a reader must perform.

4.2 Query-Relative Costs of Language Features

Because CMP is query-relative, a language feature should not be judged as simply reducing or increasing context in general. A feature can make one class of queries cheaper while making another class more expensive. The relevant question is how it changes the depth-breadth profile of the engineering queries the system realistically faces.

For example, a macro, annotation, or framework hook may reduce breadth for purpose-change queries by centralizing a repeated pattern behind one construct. The same feature may increase depth for unit-or-path understanding queries if the local code no longer reveals the generated behavior, lifecycle rule, or runtime binding that determines what happens. Similarly, typed error channels can make failure context explicit for verification and modification, while adding surface area that simple happy-path reading must still scan.

This is not a reason to treat such features as bad. Many language mechanisms are valuable precisely because they move important constraints into enforceable boundaries. Their cost appears when the boundary is not a reliable stopping point for a particular query profile - when answering the query requires reconstructing hidden expansion rules, registries, generated code, or implicit resolution paths. Under CMP, the question is whether the contexts made explicit and enforceable are the contexts that the relevant workload most needs to rely on.

4.3 Ownership as a Worked Example

Rust's ownership system is a useful concrete illustration. Without ownership, correctly using a pointer or reference may require loading substantial non-local context: who allocated it, when it is freed, whether it may be aliased, whether it crosses threads, and whether other code can mutate it concurrently. These are depth questions because the reader must traverse dependencies and conventions to reason about safety.

Ownership, borrowing, and lifetime annotations move much of this context into the type system. A reader of a function signature can answer many ownership and aliasing questions locally, and the compiler rejects many programs whose safety would otherwise depend on informal discipline. The domain complexity of memory, aliasing, and concurrency has not disappeared. It has been relocated to a boundary where callers can rely on it without reconstructing the entire usage history. This is the same pattern Parnas identified for information hiding, realized at the level of language-enforced contracts.

4.4 A CMP Reading of Language Support for Design

For software design, programming language features can be read as mechanisms for expressing and enforcing context boundaries. Structured programming constrains control-flow context. Data abstraction constrains representation context. Static type systems constrain value context.

Module systems constrain naming and reachability context. Ownership systems constrain resource-lifetime context. Effect systems constrain side-effect context.

This is not a claim that stronger enforcement is always better. Each mechanism carries its own costs in verbosity, learning burden, annotation effort, and reduced flexibility. CMP provides a way to discuss the trade-off: a language abstraction is valuable when the context it makes explicit and enforceable is greater than the context it adds through ceremony, indirection, or hidden semantics.

From this perspective, the maintainability value of programming languages lies not merely in expressiveness, but in the kinds of context they make explicit, local, and mechanically reliable for software design. The key question is how language-provided boundaries can make design abstractions more dependable.

5 AI-Assisted Engineering and Observable Context Cost

AI-assisted engineering makes context cost unusually visible. Human developers have always paid this cost through attention, navigation, working memory, review time, and missed assumptions. AI coding agents expose a more explicit trace of the same burden: retrieved files, prompt size, tool calls, localization attempts, retries, patch size, and test outcomes. A codebase that forces an agent to retrieve many unrelated files or repeatedly chase hidden conventions is making context cost visible in mechanical form.

Humans and agents use context differently. Humans rely on memory, visual scanning, social knowledge, and domain intuition; agents rely more heavily on explicit artifacts available in the prompt or tool environment. But both benefit from the same structural properties: reliable boundaries, localized domain decisions, explicit contracts, and tests that encode behavior rather than implementation accident.

AI-assisted development therefore gives CMP a practical research opportunity: it turns context minimization from a largely qualitative design intuition into something that can be studied through observable traces. The question is whether context-oriented design metrics predict better outcomes for both human and agent maintenance tasks.

6 Research Agenda

The long-term goal that motivates further work on CMP is a single one: a quantitative measure of software design quality. What can be measured can be optimized. Software design has long been discussed in qualitative terms - clarity, simplicity, maintainability - and its improvement has therefore been left to taste and experience. A faithful quantitative measure of context cost would turn design improvement into a tractable iteration: propose a change, score it, accept or revert. The questions below are the ones that stand between CMP-as-position and

CMP-as-metric: first where the open question actually lies, then what shape a candidate metric must take, and finally what stands in the way of building one.

6.1 From qualitative lens to optimization-grade resolution

The qualitative case for CMP is largely made within this paper. The principles reinterpreted in the earlier sections - information hiding, DRY, dependency inversion, architectural boundaries, ownership, and the rest - are time-tested practices, and the depth-breadth reading gives a unified account of why they help, why they conflict, and where they stop helping. That a single lens reconstructs so much accumulated design wisdom is itself evidence that it is tracking something real about design quality. The open question is therefore a quantitative one: whether a context-based *measure* can resolve depth-breadth cost finely enough to guide optimization, rather than only ratify judgments a careful designer would already make. That question is not separable from how the metric is defined: a measure too coarse to distinguish a useful abstraction from an over-engineered one will fail not because the lens is wrong, but because its quantitative form lacks resolution. Validating CMP at this level therefore has to proceed alongside concrete metric proposals, not ahead of them.

6.2 What the metric must satisfy

Before any empirical work, CMP imposes structural constraints that a candidate metric must satisfy by design. Together they describe the minimum shape a measure has to have before its quantitative resolution can even be evaluated.

It must score depth and breadth **jointly**. The two axes are not arbitrary: they correspond to the recurring query clusters CMP is built on - unit-or-path understanding and purpose change - so tracking both is also what makes the metric query-relative rather than a property of code in isolation. The Over-engineering Criterion explicitly concerns the trade-off between depth and breadth; a metric whose aggregate score improves on one axis regardless of the other is not measuring what CMP claims matters, and will mislead any optimization that targets it.

Its **resolution must be fine enough to guide optimization**. Two reasonable variants of the same module - adding a layer, splitting a responsibility, inlining a duplication, parameterizing a constant - must produce a meaningful score difference. A measure that only separates radically different codebases can ratify broad judgments but cannot drive the local design choices CMP is meant to discipline.

6.3 Open questions on the way to such a metric

How to compute depth and breadth on real code. The conceptual definitions - dependency traversal that stops at a reliable boundary, same-decision dispersion across locations - have to become procedures. Identifying where traversal can stop means modeling what a

realistic reader is entitled to rely on without crossing the boundary; identifying dispersion means recognizing when two surfaces encode the same domain decision under different syntactic forms. Both are nontrivial program analysis problems, and the metric inherits whatever resolution and noise characteristics those analyses have.

What is the workload? Because CMP is query-relative, a metric scores a design against a query distribution, not against the code in isolation. That distribution has to come from somewhere - issue history, change logs, test suites, agent traces, or curated query families standing in for representative maintenance work. Different choices yield different scores for the same code, so whether CMP measurements are comparable across systems depends on whether a defensible workload abstraction can be defined.

Goodhart-resistance under optimization. Once the metric is an optimization target, designs will be selected to improve the score. The harder question is whether they will also improve the underlying property, or whether the optimizer will find ways to lower nominal context cost without lowering real context cost - inlining behind opaque names, collapsing surfaces that hide rather than consolidate decisions, or introducing boundaries that pass syntactic checks without changing what a reader actually has to load. A usable metric has to make these gaming moves at least as expensive in score as the genuine improvements it is meant to reward.

7 Positioning

CMP sits between three neighboring bodies of work, and its contribution is clearest when stated against each in turn.

Relative to the design principles literature. The principles reinterpreted earlier in this paper - information hiding, DRY, dependency inversion, architectural boundaries, language-enforced abstractions - already articulate, in their own vocabularies, much of what CMP describes. The contribution here is not to add another principle alongside them, but to provide a single comparative frame in which they can be discussed together: each becomes a particular way of trading depth against breadth for a given query distribution. That frame is what makes it possible to ask why two principles conflict, when one principle stops helping, and what counts as over-engineering - questions that are hard to answer when each principle is held as a self-contained rule. CMP therefore stands in the lineage of Parnas’s information hiding and Ousterhout’s deep modules (Parnas 1972; Ousterhout 2018), but generalizes their core intuition - that good design lets a reader stop at a reliable boundary - into a two-axis account that also covers breadth.

Relative to formal local reasoning. Separation logic and related formal methods also place a premium on locality: they limit the heap context required to verify a piece of code (Reynolds 2002). CMP shares the locality intuition but operates at a different level. Its concern is not what can be mechanically proven about a fragment of code, but what a human or agent must

load to answer a practical engineering query - comprehension, modification, review, generation - reliably. The boundaries CMP cares about are therefore broader than those that admit formal reasoning: a type signature, a tested interface, a stable architectural seam, or an enforced ownership rule can all serve as a CMP boundary, even when no global proof obligation is in play.

Relative to traditional software metrics. Cyclomatic complexity, the CK object-oriented suite, and cognitive complexity all measure structural properties of code as proxies for maintainability (McCabe 1976; Chidamber and Kemerer 1994; Campbell 2018). CMP differs in target rather than ambition. Its quantity of interest is not a property of the code in isolation but the context a query incurs against it: what must be loaded, from where, and across which boundaries, for a given query family. From that perspective, CMP is less a competing metric than a specification of what a future metric should try to approximate; existing complexity measures can be read as partial, query-agnostic shadows of context cost.

8 Conclusion

This paper has argued that software design is best understood as the management of query-relevant context. The Context Minimization Principle holds that a design is better, all else equal, when it reduces the avoidable context a reader must load to answer the engineering queries the system realistically supports, without obscuring essential domain complexity.

Two recurring modes of context expansion - **depth**, through dependency traversal, and **breadth**, through dispersed location search - give this idea analytical traction. They give familiar design principles a common currency, explain why principles conflict, and yield the **Over-engineering Criterion**: abstraction, layering, and indirection are justified only when the breadth they remove is worth the depth they introduce for the queries the system actually faces.

The longer-term significance of the lens is that depth and breadth are structured enough to be approached by a metric, not only narrated as judgment. The value of such a metric would not be to replace design taste, but to make design improvement *optimizable* - to turn it into an iteration loop in which a change to the code produces a measurable change in score. CMP is offered here not as that metric, but as the conceptual ground on which one might be built.

Beck, Kent. 2002. *Test-Driven Development: By Example*. Addison-Wesley Professional.

Brooks, Frederick P. 1987. “No Silver Bullet: Essence and Accidents of Software Engineering.” *Computer* 20 (4): 10–19. <https://doi.org/10.1109/MC.1987.1663532>.

Campbell, G. Ann. 2018. “Cognitive Complexity: An Overview and Evaluation.” *Proceedings of the 2018 International Conference on Technical Debt*, 57–58. <https://doi.org/10.1145/3194164.3194186>.

- Chidamber, Shyam R., and Chris F. Kemerer. 1994. “A Metrics Suite for Object Oriented Design.” *IEEE Transactions on Software Engineering* 20 (6): 476–93. <https://doi.org/10.1109/32.295895>.
- Cockburn, Alistair. 2005. *Hexagonal Architecture*. <https://alistair.cockburn.us/hexagonal-architecture/>.
- Evans, Eric. 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Martin, Robert C. 2017. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall.
- McCabe, Thomas J. 1976. “A Complexity Measure.” *IEEE Transactions on Software Engineering* SE-2 (4): 308–20. <https://doi.org/10.1109/TSE.1976.233837>.
- Metz, Sandi. 2016. *The Wrong Abstraction*. <https://sandimetz.com/blog/2016/1/20/the-wrong-abstraction>.
- Ousterhout, John. 2018. *A Philosophy of Software Design*. Yaknyam Press.
- Parnas, David L. 1972. “On the Criteria to Be Used in Decomposing Systems into Modules.” *Communications of the ACM* 15 (12): 1053–58. <https://doi.org/10.1145/361598.361623>.
- Reynolds, John C. 2002. “Separation Logic: A Logic for Shared Mutable Data Structures.” *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science*, 55–74. <https://doi.org/10.1109/LICS.2002.1029817>.
- Shannon, Claude E. 1948. “A Mathematical Theory of Communication.” *The Bell System Technical Journal* 27 (3): 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.